AdaTec · Coding Tank · Projeto Final · Grupo 3

RELATÓRIO TÉCNICO

OrderMind

Sistema inteligente de decisão sobre pedidos

Motor de priorização: Marketplace Priority Sorter (MPS) — especificação do algoritmo, análise de desempenho e plano de validação

Sistema	OrderMind — sistema inteligente de decisão sobre pedidos
Cliente	VelozMart — logística e varejo, América Latina
Documento	Especificação técnica do motor de priorização (MPS) do OrderMind para pedidos aguardando expedição
Versão	2.0 (consolidada)
Data	15 de outubro de 2026
Equipe responsável	Adriano Ferreira Cardoso · Julia Neves Oliveira · Lucas Augusto Vieira Leão Raniele Albertim Vieira da Silva Alves · Wanderley Damazio Gonçalves
Apresentação interativa	apresentacao-adatec.netlify.app

Cenário fictício elaborado para o Projeto Final do Coding Tank — Ordenação e Prioridade. Os resultados de simulação não são dados reais da VelozMart.

Sumário

Resumo executivo . . . 3

1. Problema . . . 4

2. Algoritmo e estrutura de dados . . . 5

3. Diagramas do processo . . . 8

4. Justificativa dos critérios de ordenação . . . 11

5. Análise Big O e comparação de abordagens . . . 12

6. Simulação e teste A/B . . . 13

7. Mapa de trade-offs e riscos . . . 15

8. Acessibilidade na interface . . . 16

9. Próximos passos e conclusão . . . 16

A. Apêndice – Exemplo numérico do score . . . 17

B. Apêndice – Glossário . . . 17

C. Apêndice – Apresentação interativa . . . 18

Como ler este documento

As Seções 1 a 3 definem o problema, o algoritmo (com pseudocódigo) e os diagramas, incluindo o fluxograma. As Seções 4 e 5 justificam as escolhas e analisam a complexidade. A Seção 6 apresenta os resultados de simulação e o plano de teste A/B. As Seções 7 a 9 tratam de trade-offs, riscos, acessibilidade e próximos passos. Os apêndices trazem exemplos numéricos, o glossário e o acesso à apresentação interativa.

Resumo executivo

O **OrderMind** é um sistema inteligente de decisão sobre pedidos, desenvolvido para a VelozMart. Seu motor de priorização, o **Marketplace Priority Sorter (MPS)**, determina a ordem de expedição dos pedidos por meio de um **heap binário máximo** ordenado por um **score composto** que combina três variáveis: o **priorityScore** (calculado por IA), o **dispatchWindow** (minutos até o prazo expirar) e o **sizeCategory** (P, M ou G, que define o espaço ocupado na doca de embalagem). O topo do heap é sempre o pedido mais valioso de despachar naquele instante.

O documento responde às três questões formuladas pela VelozMart: (I) como ordenar por múltiplos critérios, com a fórmula de score e seus pesos configuráveis; (II) como re-enfileirar quando chegam pedidos novos ou os prazos mudam, com quatro operações de custo conhecido; e (III) como comprovar a vantagem sobre a ordenação simples por tempo, com simulação e teste A/B.

Indicador (simulação, semente 42, turno de 8 h)	FIFO	MPS	Variação
Atraso médio	8,7 min	1,6 min	−81,6% (≈ 5× menos)
Pedidos dentro do prazo	67%	85%	≈ +17 p.p.
Throughput por hora	34,5 pedidos	36,0 pedidos	+4,3%

Tabela 1 – Resultados principais. Simulação com dados sintéticos e a mesma doca de 4 bancadas para todas as estratégias.

Leitura crítica

O peso composto simples (priorityScore × 1/dispatchWindow) praticamente empata com o MPS no atraso médio (1,5 min contra 1,6 min). A vantagem do MPS está em entregar mais pedidos no prazo e mais throughput, e em permitir que a operação calibre os pesos sem alterar o heap. Os resultados vêm de uma única semente de simulação e devem ser confirmados com replays de dados reais e teste A/B (Seção 6).

1. Problema

A VelozMart atua em logística e varejo na América Latina e processa milhares de pedidos por dia. Cada pedido aprovado entra em uma fila de expedição, aguardando embalagem e envio. A questão central é: **em que ordem expedir para minimizar atrasos e custos logísticos?**

Atributo	Descrição	Impacto na prioridade
priorityScore	Pontuação de 0 a 100 calculada por IA a partir de distância, SLA, valor do item e reputação do vendedor.	Quanto maior, maior a prioridade.
dispatchWindow	Minutos restantes até o prazo de expedição expirar; diminui a cada minuto de espera.	Quanto menor, mais urgente.
sizeCategory	P, M ou G. Define o espaço ocupado na doca de embalagem (neste protótipo: 1, 2 e 4 de 4 bancadas).	Afeta o encaixe na doca e o throughput.

Tabela 2 – Atributos do pedido e impacto na priorização

Questões a serem respondidas

1. **Como ordenar?** Combinar múltiplos critérios em uma única ordem de saída.
2. **Como re-enfileirar?** Novos pedidos chegam e os prazos mudam o tempo todo.
3. **Como provar?** Métricas que mostrem ganho sobre a ordenação simples por tempo.

Abordagens consideradas

Abordagem	Vantagem	Limitação
Peso composto: priorityScore × 1/dispatchWindow	Simples de calcular.	Ignora o tamanho; a divisão é indefinida quando a janela chega a zero.
Duas filas (urgentes e normais)	Fácil de operar.	Cria starvation na fila normal e depende de um limiar fixo.
Scheduling por prazo (EDF adaptado)	Prioriza o que vence primeiro.	Ignora valor do pedido e tamanho.
Heap multi-chave (escolhida)	Estrutura dedicada, comparação composta, custo logarítmico por evento.	Exige calibrar pesos e é mais complexa que uma lista.

Tabela 3 – Abordagens avaliadas para a priorização

2. Algoritmo e estrutura de dados

A solução adota um **max-heap binário** como estrutura central: cada nó contém um pedido e o seu score composto, e a raiz (índice 0 do array) é sempre o pedido de maior score. Os filhos do nó *i* ficam nas posições 2i+1 e 2i+2. O score depende do tempo, pois o dispatchWindow decresce; assim, pedidos que se aproximam do prazo são promovidos automaticamente.

2.1 Fórmula de priorização

Score composto

score(o, t) = 100 · (α · IA + β · U + γ · S), com γ = 1 − α − β

Os três pesos sempre somam 1 e são configuráveis: a operação decide o que priorizar sem alterar o código do heap.

Termo	Definição	Faixa
IA	priorityScore ÷ 100: valor, SLA, distância e reputação trazidos pela IA.	0 a 1
U	Urgência hiperbólica: 1 ÷ (1 + dispatchWindow ÷ 15); vale 1 se o pedido já venceu. Sobe devagar quando há folga e dispara perto do prazo. Generaliza 1/dispatchWindow sem a divisão por zero e impede starvation, pois todo pedido acaba ficando urgente.	0 a 1
S	Fator de tamanho: 1,0 (P), 0,6 (M) ou 0,3 (G). Pedidos pequenos ocupam menos bancadas e liberam a doca mais cedo.	0,3 a 1

Tabela 4 – Componentes do score (a constante de escala da urgência é 15 minutos no protótipo)

Peso	Significado	Padrão	Justificativa e métrica afetada
α	priorityScore (IA)	0,30	Aproveita distância, SLA, valor e reputação; protege receita e experiência. Afeta custo logístico e % no prazo.
β	Urgência (dispatchWindow)	0,40	Prioriza quem está perto do prazo e evita starvation. Afeta o atraso médio.
γ	Espaço na doca (tamanho)	0,30	Pedidos pequenos liberam a doca mais cedo. Afeta throughput por hora e uso da doca.

Tabela 5 – Pesos configuráveis (valores padrão do protótipo)

2.2 Tamanho e encaixe na doca

A doca de embalagem tem 4 bancadas. Cada categoria ocupa 1 (P), 2 (M) ou 4 (G) bancadas durante 2, 3 ou 5 minutos de serviço, respectivamente. Quando uma bancada libera, o sistema retira o topo do heap; se o pedido não couber nas bancadas livres, ele é guardado e o sistema tenta o próximo candidato, até 8 (LOOKAHEAD). Os pedidos pulados voltam ao heap ao final da tentativa.

2.3 Operações e custo

Operação	Descrição	Custo
Inclusão	Calcula o score e insere no final do array; sift-up restaura a propriedade de max-heap.	O(log n)
Passagem do tempo	A cada minuto as janelas encolhem, os scores são recalculados e o heap é reconstruído (heapify).	O(n)
Prazo alterado	Recalcula só o score do pedido e o reposiciona (update-key).	O(log n)*
Expedição	Retira o topo; se não couber na doca, guarda e tenta o próximo (K ≤ 8) e devolve os pulados.	O(K · log n)
Consulta (peek)	Exibe o próximo pedido sem removê-lo.	O(1)
Cancelamento (previsto)	Localiza o pedido pelo índice e o remove, restaurando o heap.	O(log n)*

*Tabela 6 – Operações do MPS. * Com índice id → posição; no protótipo a posição é achada por busca linear, O(n).*

2.4 Pseudocódigo

Parâmetros e função de score:

```
alpha, beta ← 0.30, 0.40;  gamma ← 1 − alpha − beta        // pesos configuráveis
pesoDoca ← {P: 1.0, M: 0.6, G: 0.3}
bancadas ← {P: 1,   M: 2,   G: 4};  LOOKAHEAD ← 8

score(o, t):
    IA ← o.priorityScore / 100
    U  ← 1                                   se o.dispatchWindow ≤ 0
    U  ← 1 / (1 + o.dispatchWindow / 15)     caso contrário
    S  ← pesoDoca[o.sizeCategory]
    retorne 100 · (alpha·IA + beta·U + gamma·S)
```

Eventos do sistema:

```
ao_chegar(o):                                       // inclusão
    o.score ← score(o, agora)
    insert(heap, o)                                 // sift-up: O(log n)

a_cada_minuto():                                    // passagem do tempo
    para cada p em heap:
        p.dispatchWindow ← p.dispatchWindow − 1
        p.score ← score(p, agora)
    heapify(heap)                                   // buildHeap: O(n)

prazo_mudou(o):                                     // re-priorização pontual
    o.score ← score(o, agora)
    update(heap, o)                                 // sobe ou desce: O(log n)*

doca_livre():                                       // expedição
    pulados ← []
    repita até LOOKAHEAD vezes:
        o ← extractMax(heap)                        // O(log n)
        se bancadas[o.sizeCategory] ≤ bancadasLivres:
            expedir(o);  pare
        senão: pulados.add(o)
    reinsira(pulados)                               // devolve ao heap
```

Operações internas do heap:

```
maior(a, b):                                         // comparação do heap
    se a.score ≠ b.score: retorne a.score > b.score
    retorne a.chegada < b.chegada                    // desempate (proposta)

sift-up(i):    enquanto i > 0 e maior(h[i], h[pai(i)]): trocar; i ← pai(i)
sift-down(i):  repita:
                   m ← o maior entre i e seus filhos 2i+1 e 2i+2
                   se m = i: pare
                   trocar(i, m);  i ← m
extractMax():  r ← h[0]; h[0] ← último; remover último; sift-down(0); retorne r
update(h, o):  i ← posição(o); sift-up(i); sift-down(i)
heapify(h):    para i de ▫n/2▫−1 até 0: sift-down(i)                // O(n)
// * posição(o) em O(1) com índice id → posição; no protótipo, busca linear.
```

O desempate por chegada na função maior é uma proposta deste relatório: a especificação original não define a regra para scores iguais, e ela torna a ordem previsível.

3. Diagramas do processo

3.1 Fluxograma do sistema

O fluxograma resume o funcionamento do MPS: três tipos de evento alimentam o heap e a doca consome o pedido de maior score que caiba nas bancadas livres.

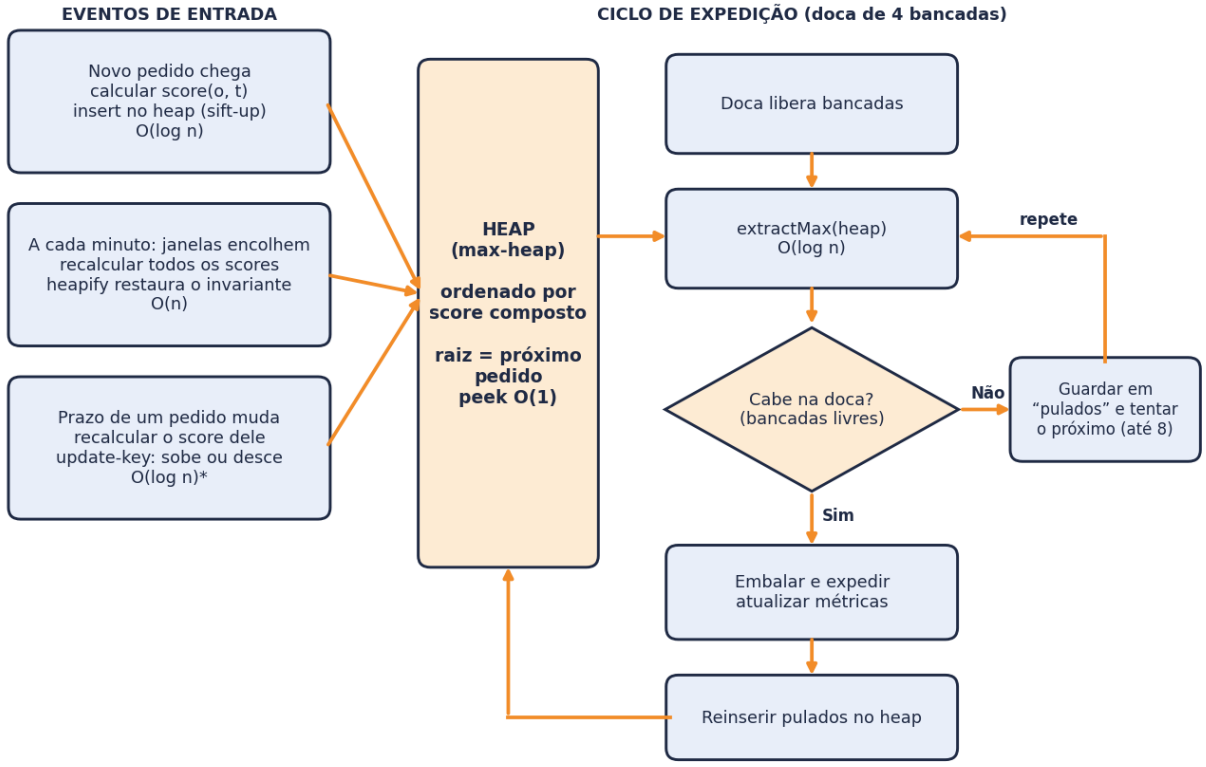


*Figura 1 – Fluxograma do MPS. A cada minuto, os scores são recalculados (heapify, O(n)); a doca tenta receber o pedido de maior score que caiba, examinando até 8 candidatos. * Com índice id → posição.*

3.2 Fluxo por tipo de evento

Evento	Passo 1	Passo 2	Passo 3	Custo
Inclusão	Novo pedido chega	Calcula α·IA + β·U + γ·S	insert no heap (sift-up)	O(log n)
Passagem do tempo	Cada minuto: janelas encolhem	Recalcula todos os scores	heapify restaura o invariante	O(n)
Prazo alterado	Evento muda o prazo de um pedido	Recalcula o score dele	update-key: sobe ou desce	O(log n)*
Expedição	Doca tem bancada livre	extract-max (topo do heap)	Cabe? Sim: embala. Não: tenta o próximo; devolve os pulados	O(K · log n)

*Tabela 7 – Quatro eventos, um único heap. K = candidatos examinados (8 no protótipo). * Com índice id → posição.*

3.3 Estrutura do heap

O heap é representado como um array. A tabela abaixo mostra um exemplo com 7 pedidos, no formato exibido pela apresentação interativa (pedido · tamanho).

Posição	Pedido	Score	Pai	Filhos
0 (raiz)	#6 · P	60	—	1, 2
1	#4 · P	52	0	3, 4
2	#1 · P	55	0	5, 6
3	#2 · P	39	1	—
4	#5 · P	43	1	—
5	#3 · P	51	2	—
6	#7 · M	53	2	—

Tabela 8 – Heap de 7 pedidos como array (filhos de i: 2i+1 e 2i+2; máximo no índice 0)

3.4 Inclusão, remoção e re-priorização

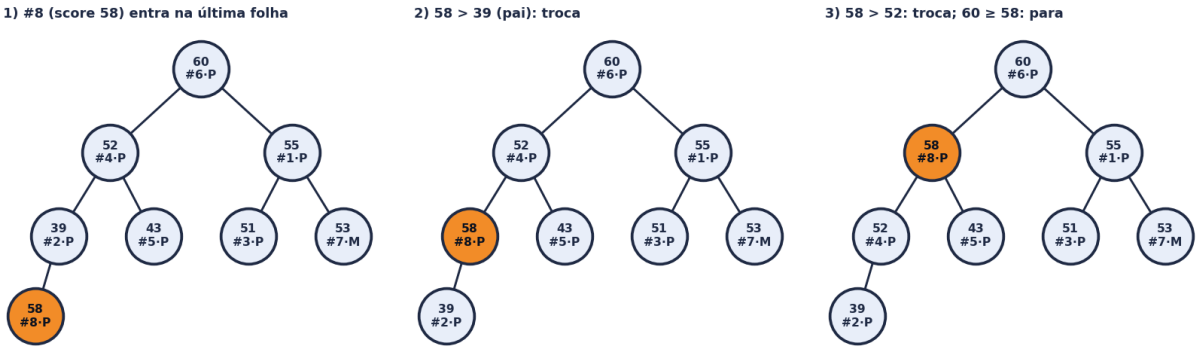


Figura 2 – Inclusão e sift-up. O pedido novo #8 (score 58) entra na última folha, troca com o pai (39) e depois com o seguinte (52); a propagação termina abaixo da raiz (60), que é maior.

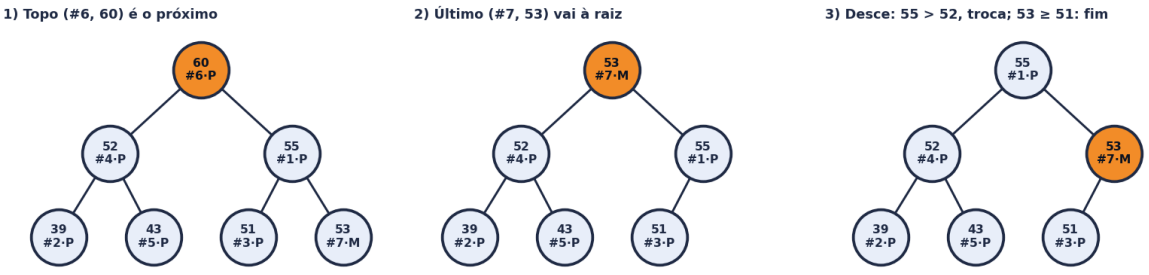


Figura 3 – Expedição do topo e sift-down. O topo (#6, 60) segue para a doca; o último nó (#7, 53) ocupa a raiz e desce, trocando com o maior filho (55), até restaurar a propriedade do heap.

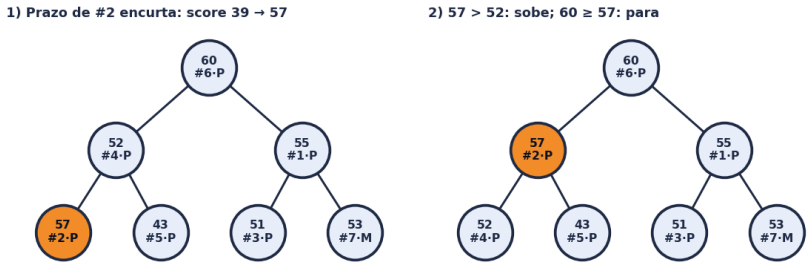


Figura 4 – Re-priorização por prazo (update-key). O prazo do pedido #2 encurta e seu score sobe de 39 para 57; ele sobe trocando com o pai (52) e para abaixo da raiz (60).

3.5 Ocupação da doca (Gantt)

O diagrama abaixo ilustra a regra de encaixe: cada bloco é um pedido ocupando bancadas (B1 a B4) pelo tempo de serviço da sua categoria.

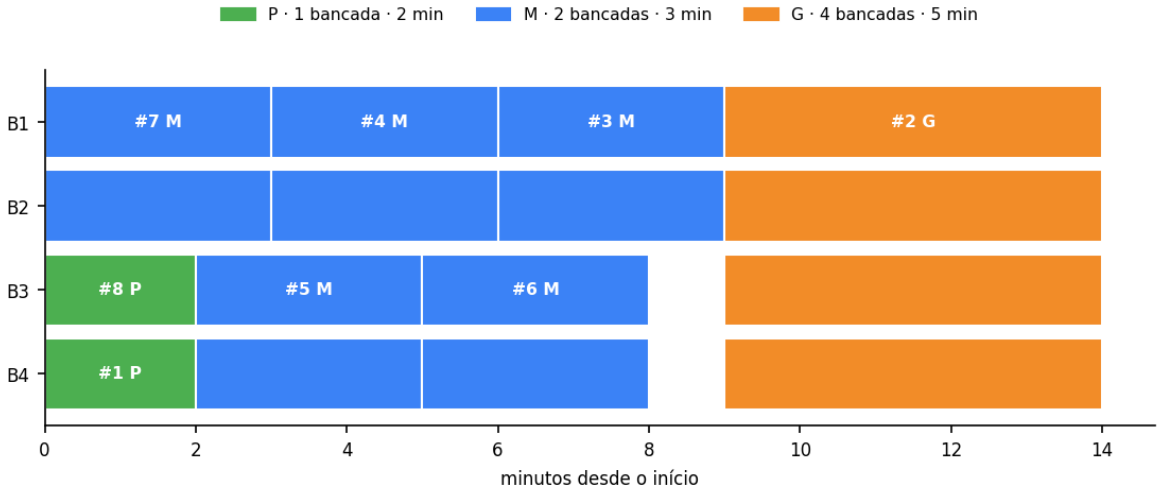


Figura 5 – Gantt ilustrativo da doca, construído com a ordem de prioridade da tela "Fila ao vivo" da apresentação (#7, #8, #1, #2, #5, #4, #6, #3) e a regra de encaixe com até 8 candidatos. Não é saída direta do simulador. O pedido G (#2) só inicia aos 9 min, quando as 4 bancadas ficam livres ao mesmo tempo (ver risco na Seção 7). O Gantt completo FIFO × MPS está na apresentação interativa.

4. Justificativa dos critérios de ordenação

Uma ordenação baseada apenas em tempo (FIFO ou prazo) ignora o valor estratégico do pedido: um pedido de alto valor para um cliente premium pode ter prazo confortável e, ainda assim, representar maior impacto comercial. O score composto equilibra **urgência** (prazo), **importância** (score da IA) e **eficiência operacional** (tamanho).

Critério	Componente	Papel na decisão	Efeito esperado
priorityScore	α · IA	Incorpora valor, SLA, distância e reputação já calculados pela IA.	Atender pedidos relevantes com menor atraso; reduzir custo logístico.
dispatchWindow	β · U	Cresce de forma hiperbólica e dispara perto do prazo; impede starvation.	Reduzir pedidos vencidos e o atraso médio; proteger o SLA.
sizeCategory	γ · S	Pedidos pequenos liberam a doca mais cedo e permitem atender vários ao mesmo tempo.	Aumentar o throughput por hora e o uso da doca.

Tabela 9 – Critérios, componentes do score e efeito esperado

Como o tamanho entra com peso (γ) e não apenas como desempate, pedidos grandes continuam sendo priorizados pela urgência e pelo valor: um pedido G de IA alta ou com prazo curto ainda alcança o topo.

Efeitos esperados nas métricas

- Menor atraso médio
- Menor percentual de pedidos vencidos e maior percentual de SLA cumprido
- Maior throughput por hora
- Menor atraso médio dos pedidos com score ≥ 80

5. Análise Big O e comparação de abordagens

Custo por evento para n = 50.000 pedidos pendentes. As colunas de operações são **estimativas teóricas**.

Estrutura ou estratégia	Complexidade e estimativa	Leitura
Heap binário (escolhido)	Inserir O(log n) ≈ 16 passos; extrair O(K · log n) com K ≤ 8; atualizar O(log n) com índice; consultar topo O(1); memória O(n).	Adequado a eventos dinâmicos e ao acesso direto ao próximo pedido.
Lista ordenada (inserção)	Inserir O(n) ≈ 25.000 deslocamentos em média; extrair O(1) ou O(n).	Só compensa para listas pequenas ou quase ordenadas.
Reordenar tudo (Merge Sort)	O(n log n) ≈ 780.000 comparações por evento.	Estável, mas exige O(n) de memória extra.
Reordenar tudo (Heap Sort)	O(n log n) por evento.	In-place, porém não estável.
Selection Sort	O(n^2) ≈ 2,5 bilhões de operações.	Inviável nessa escala.
Ordenação simples por tempo (FIFO)	Fila: O(1) por evento.	Barata, porém ignora score e tamanho.

Tabela 10 – Comparação de custo por evento (n = 50.000)

Reavaliação por passagem do tempo

Como o score depende do tempo, a cada minuto todos os scores são recalculados. Reconstruir o heap custa O(n), da ordem de 100.000 comparações para n = 50.000, contra ≈ 780.000 de uma reordenação completa O(n log n): cerca de 8 vezes menos. Em ambos os casos o custo é pequeno frente à janela de um minuto.

Medição complementar

Para ancorar a teoria, medimos o tempo por evento (inserir um pedido e retirar o próximo) em Python 3, com n = 50.000 e 300 eventos por abordagem:

Abordagem	Tempo por evento	Em relação ao heap
Heap binário	0.0006 ms	1×
Lista ordenada (inserção)	0.0138 ms	≈ 23× mais lento
Reordenar tudo a cada evento	0.2901 ms	≈ 484× mais lento

Tabela 11 – Medição complementar. A ordenação do Python aproveita listas quase ordenadas, o que favorece a reordenação; com um Merge Sort puro a diferença seria maior.

O fluxo é dinâmico, há muitas chegadas e mudanças e apenas o próximo pedido importa. Manter toda a fila ordenada é desperdício; o heap realiza apenas o trabalho necessário.

6. Simulação e teste A/B

6.1 Simulação

A apresentação interativa (Vue.js 3) simula um turno de 8 horas com carga alta, **semente 42**, e a **mesma doca de 4 bancadas** para as três estratégias comparadas:

- **FIFO**: ordenação simples por tempo de chegada (linha de base).
- **Composto**: peso composto simples, priorityScore × 1/dispatchWindow.
- **MPS**: heap multi-chave com α, β e γ (padrão 0,30 / 0,40 / 0,30).

Métrica	FIFO	Composto	MPS (heap)	MPS vs FIFO
Atraso médio	8,7 min	1,5 min	1,6 min	−81,6% (≈ 5×)
Pedidos no prazo	67%	n/d	85%	≈ +17 p.p.
Throughput/hora	34,5 pedidos	n/d	36,0 pedidos	+4,3%

Tabela 12 – Métricas comparativas (média da simulação). n/d: não reportado para a estratégia Composto. Percentuais exibidos arredondados.

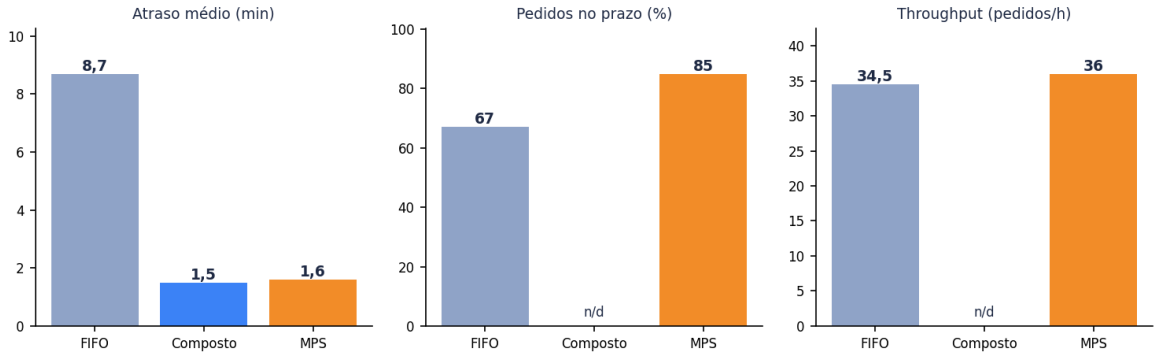


Figura 6 – Comparação das três estratégias: atraso médio (menor é melhor), pedidos no prazo e throughput (maior é melhor).

Leitura crítica dos resultados

- No atraso médio, o peso composto simples praticamente empata com o MPS (1,5 contra 1,6 min). O MPS entrega mais pedidos no prazo e mais throughput, além de permitir calibrar α, β e γ.
- Os valores dependem da evolução do turno: durante a execução (por exemplo, às 3 h 36) os painéis exibem números parciais diferentes dos do fechamento de 8 h usado neste relatório.
- Uma única semente não mede a variabilidade. Recomenda-se repetir a simulação com várias sementes e reportar média e desvio.
- São dados sintéticos, não dados reais da VelozMart.

6.2 Proposta de teste A/B

Em produção, dividir metade dos centros de distribuição para cada estratégia durante duas semanas, mantendo a mesma regra de encaixe na doca, com monitoramento de segurança operacional e calibração de α, β e γ. Compara-se a linha de base (ordenação por tempo) com o heap composto.

Métrica	Como comparar	Objetivo de leitura
Atraso médio	Média entre a linha de base e o heap composto.	Buscar redução consistente.
% de pedidos vencidos	Parcela de pedidos que ultrapassa a janela.	Evitar aumento relevante.
Throughput/hora	Pedidos processados por hora.	Preservar ou ampliar capacidade.
Atraso dos pedidos prioritários	Isolar os pedidos com score ≥ 80.	Proteger pedidos prioritários.
Tempo de resposta por operação	Medir inserir, atualizar, retirar e consultar topo.	Manter latência operacional aceitável.

Tabela 13 – Métricas e critérios de leitura do teste A/B

Critérios de leitura

- Melhoria consistente sem aumento relevante de pedidos vencidos
- Latência operacional aceitável
- Ganho ou neutralidade de throughput

7. Mapa de trade-offs e riscos

7.1 Simplicidade × desempenho × manutenibilidade

Estratégia	Simplicidade	Desempenho	Manutenibilidade
Lista simples	Alta	Baixo a moderado	Alta
Reordenação completa	Moderada	Baixo em eventos frequentes	Moderada
Heap binário	Moderada	Alta para o próximo pedido	Alta, com índice id → posição

Tabela 14 – Mapa conceitual de trade-offs

O heap foi escolhido pelo equilíbrio entre complexidade de implementação e desempenho: para um sistema que processa dezenas de milhares de pedidos por hora, inserção e remoção em O(log n) são essenciais, e a re-priorização em O(n) permite adaptação em tempo real às mudanças de prazo, algo inviável com FIFO estático.

Prós	Contras
Desempenho — O(log n) por evento.	**Complexidade estrutural** — mais complexo que uma lista.
Adaptação em tempo real — urgência hiperbólica promove pedidos automaticamente.	**Calibração** — α, β e γ exigem justificativa de negócio.
Disponibilidade — estrutura clássica presente em bibliotecas.	**Visão da fila** — o heap dá acesso direto apenas ao próximo pedido; listar a fila em ordem custa O(n log n).

Tabela 15 – Prós e contras da solução

7.2 Riscos e mitigações

Risco	Mitigação proposta
Adiamento de pedidos G pelo encaixe com lookahead: bancadas parciais são preenchidas por pedidos menores e o pedido G só inicia quando as 4 bancadas ficam livres (Figura 5).	Reservar a doca para o topo após um número máximo de pulos, ou limitar o tempo de espera; monitorar o atraso dos pedidos G.
Busca linear no update-key do protótipo, O(n).	Implementar o heap indexado (id → posição) para O(log n).
Resultados com uma única semente.	Replicar com várias sementes, replay de dados reais e teste A/B.
Scores iguais sem regra de desempate definida.	Desempatar pelo menor instante de chegada.
Reordenação visual brusca da lista a cada minuto.	Animar com moderação e permitir pausar atualizações na interface.

Tabela 16 – Riscos identificados e mitigações

8. Acessibilidade na interface

A interface do painel de expedição deve garantir uso por todos, seguindo diretrizes baseadas na WCAG 2.1:

- **Contraste alto** em textos e indicadores sobre o fundo.
- **Não depender apenas de cor:** a urgência é comunicada por ícone e texto explícito (“Urgente”).
- **Navegação por teclado:** todos os controles (botões e sliders de peso) acessíveis por Tab e Enter.
- **Atualizações em tempo real** anunciadas por região *aria-live*, sem excesso de alertas.
- **Diagramas com texto alternativo:** a árvore do heap e o Gantt descritos textualmente.

Status	Como é comunicado
✓ **Normal**	Texto e ícone neutros; a leitura não depende de cor.
⚠ **Urgente**	Marcado explicitamente por ícone e pela palavra “Urgente”.
■ **Em espera**	Estado compreensível mesmo sem perceber cores.

Tabela 17 – Amostra de status e comunicação sem depender de cor

9. Próximos passos e conclusão

Próximos passos

1. **Implementar o heap indexado** (id → posição) para update-key e cancelamento em O(log n).
2. **Replay com dados reais** da VelozMart e simulações com várias sementes.
3. **Teste A/B** de duas semanas, por centro de distribuição.
4. **Calibrar α, β e γ** com dados reais e custo de atraso por tipo de pedido.
5. **Tratar o adiamento de pedidos G** com reserva de doca ou limite de espera.
6. **Ajustar o score com machine learning** e incorporar a capacidade da doca por tamanho.

Conclusão

O MPS combina um heap binário com uma fórmula de priorização multi-critério e entrega uma solução robusta para a ordenação de pedidos em tempo real. O custo é O(log n) por inclusão e por alteração de prazo, O(n) por minuto para a reavaliação pelo tempo e O(K · log n) por expedição, com $K \leq 8$. Na simulação, equilibra urgência, valor e eficiência operacional, com ≈ 5× menos atraso médio, +17 p.p. de pedidos no prazo e +4,3% de throughput frente ao FIFO, resultados a confirmar com dados reais.

Critérios de sucesso

- Decisão rápida
- SLA protegido
- Pedidos prioritários atendidos com menor atraso
- Throughput preservado ou ampliado
- Operação acessível

A. Apêndice – Exemplo numérico do score

Com α = 0,30, β = 0,40 e γ = 0,30. O pedido A (IA 85, prazo de 12 min, tamanho P) obtém 100 × (0,30 × 0,85 + 0,40 × 0,556 + 0,30 × 1,0) ≈ 78; o pedido B (IA 50, prazo de 90 min, tamanho G) obtém ≈ 30. Logo, A sai primeiro, ainda que B tenha chegado antes. Os pedidos #7, #8, #1 e #2 reproduzem os valores exibidos na tela “Fila ao vivo” da apresentação.

Pedido	priorityScore	Janela	Tam.	IA	U	S	Score
A	85	12 min	P	0,85	0,556	1,0	77,7
B	50	90 min	G	0,50	0,143	0,3	29,7
#7	96	21 min	M	0,96	0,417	0,6	63,5
#8	61	27 min	P	0,61	0,357	1,0	62,6
#1	28	58 min	P	0,28	0,205	1,0	46,6
#2	92	84 min	G	0,92	0,152	0,3	42,7

Tabela 18 – Cálculo do score composto (valores conferidos com a fórmula)

B. Apêndice – Glossário

Termo	Significado
SLA	Prazo combinado com o cliente para entregar o pedido.
Throughput	Quantidade de pedidos que saem por hora.
Heap / max-heap	Fila em forma de pirâmide (árvore binária); o pedido de maior score fica no topo.
sift-up / sift-down	Troca de um nó com o pai (sobe) ou com o maior filho (desce) para restaurar a propriedade do heap.
heapify	Reconstrução do heap a partir de um array desordenado, em O(n).
update-key	Alteração do score de um pedido e reposicionamento na árvore.
FIFO	“Primeiro a chegar, primeiro a sair”.
Time decay	O tempo passa e o prazo fica mais perto: o pedido “envelhece” e ganha prioridade.
Starvation	Espera indefinida de um pedido que sempre é ultrapassado por outros.
Doca	Bancadas de embalagem do galpão onde o pedido é preparado.

Tabela 19 – Glossário

C. Apêndice – Apresentação interativa

Uma apresentação interativa em Vue.js 3 complementa este documento. Ela permite visualizar o heap se ajustando a cada inclusão e remoção, comparar FIFO, Composto e MPS ao vivo, ajustar os pesos α, β e γ com sliders e simular o decaimento temporal dos prazos.

Acesse (abra no navegador): https://apresentacao-adatec.netlify.app